

Python Programming

100+ Programs: Basic to Advanced

Submitted by: **Prajyot Bhadange**

PG Diploma in HPC System Administration (PGDHPCSA)

CDAC ACTS, Pune

Subject: Python Programming

Total Programs: 125

Table of Contents

Basics

1. Hello World
2. Variables and Data Types
3. Taking Multiple Values (simulated input)
4. Arithmetic Operators
5. Relational and Logical Operators
6. Type Conversion
7. Swapping Two Numbers
8. Simple Calculator
9. Area and Perimeter of Rectangle
10. Simple Interest Calculator

Control Flow

11. Check Even or Odd
12. Largest of Three Numbers
13. Leap Year Checker
14. Number Pattern using Nested Loops
15. Pyramid Pattern
16. While Loop - Sum of Digits
17. Break and Continue Demo
18. Multiplication Table

Functions & Recursion

19. Function with Default Arguments
20. Variable Arguments (*args, **kwargs)
21. Factorial using Recursion
22. Fibonacci using Recursion
23. Sum of Natural Numbers (Recursion)
24. GCD using Recursion (Euclidean Algorithm)
25. Power Function using Recursion
26. Tower of Hanoi
27. Palindrome Check using Recursion
28. Higher Order Function Example

Strings

29. String Reversal
30. Palindrome String Check
31. Count Vowels and Consonants
32. String Formatting Methods
33. Word Frequency Counter
34. Anagram Checker
35. Remove Duplicate Characters
36. Capitalize Each Word (Title Case)
37. String Slicing Demo
38. Longest Word in a Sentence

Data Structures (Built-in)

39. List Operations
40. List Comprehension - Squares
41. Nested List (Matrix) Operations
42. Tuple Basics and Immutability
43. Dictionary Operations

- 44. Set Operations
- 45. Merge Two Dictionaries
- 46. List of Dictionaries - Sort by Key
- 47. Find Duplicate Elements in a List
- 48. Counter from collections
- 49. defaultdict Grouping Example
- 50. Zip Function - Combine Two Lists

File Handling

- 51. Write and Read a Text File
- 52. Append to a File
- 53. Count Lines, Words, Characters in a File
- 54. Read File Line by Line with Line Numbers
- 55. Working with CSV Files
- 56. Working with JSON Files

Exception Handling

- 57. Basic Try-Except
- 58. Multiple Except Blocks
- 59. Try-Except-Else-Finally
- 60. Custom Exception Class
- 61. Handling Multiple Exceptions in a Loop
- 62. Assertions in Python

Object Oriented Programming

- 63. Class and Object Basics
- 64. Class Variables vs Instance Variables
- 65. Inheritance Example
- 66. Multiple Inheritance
- 67. Encapsulation with Private Variables
- 68. Polymorphism with Function Overriding
- 69. Abstract Base Class
- 70. Operator Overloading
- 71. Static Method and Class Method
- 72. Property Decorator (Getter/Setter)
- 73. Duck Typing Demonstration
- 74. Composition Example (Has-A Relationship)

Iterators, Generators & Decorators

- 75. Custom Iterator Class
- 76. Generator Function - Fibonacci Sequence
- 77. Generator Expression
- 78. Simple Function Decorator
- 79. Decorator with Arguments
- 80. Timing Decorator
- 81. Context Manager using Class
- 82. Context Manager using contextlib

Functional Programming

- 83. Lambda Functions
- 84. Map Function
- 85. Filter Function
- 86. Reduce Function
- 87. functools.lru_cache for Memoization
- 88. itertools.permutations and combinations
- 89. itertools.groupby

90. Partial Functions with functools

Standard Library Modules

- 91. Math Module Basics
- 92. Random Module (with fixed seed)
- 93. Datetime Module
- 94. Regular Expressions - Pattern Matching
- 95. Regular Expressions - Validate Phone Number
- 96. OS Module - File System Info
- 97. Working with Collections - namedtuple

Sorting Algorithms

- 98. Bubble Sort
- 99. Selection Sort
- 100. Insertion Sort
- 101. Merge Sort
- 102. Quick Sort
- 103. Counting Sort

Searching Algorithms

- 104. Linear Search
- 105. Binary Search (Iterative)
- 106. Binary Search (Recursive)

Data Structures & Algorithms

- 107. Stack Implementation
- 108. Queue Implementation
- 109. Singly Linked List Implementation
- 110. Binary Search Tree - Insert and Inorder Traversal
- 111. Graph - BFS Traversal
- 112. Graph - DFS Traversal

Dynamic Programming

- 113. Fibonacci using Dynamic Programming (Tabulation)
- 114. 0/1 Knapsack Problem
- 115. Longest Common Subsequence
- 116. Coin Change Problem (Minimum Coins)

Advanced Python Concepts

- 117. Closures in Python
- 118. *args, **kwargs in Class Constructor
- 119. Multithreading Example
- 120. Multiprocessing Example
- 121. Async/Await Example
- 122. Metaclass Example
- 123. Data Classes (Python 3.7+)
- 124. Enum Class Example
- 125. Type Hints and mypy-style Annotations

Basics

1. Hello World

CODE

```
print("Hello, World!")
```

OUTPUT

```
Hello, World!
```

2. Variables and Data Types

CODE

```
name = "Prajyot"  
age = 25  
height = 5.9  
is_student = True  
print(name, age, height, is_student)  
print(type(name), type(age), type(height), type(is_student))
```

OUTPUT

```
Prajyot 25 5.9 True  
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
```

3. Taking Multiple Values (simulated input)

CODE

```
a, b = 10, 20  
print("a =", a)  
print("b =", b)  
print("Sum =", a + b)
```

OUTPUT

```
a = 10  
b = 20  
Sum = 30
```

4. Arithmetic Operators

CODE

```
a, b = 15, 4
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor Division:", a // b)
print("Modulus:", a % b)
print("Exponent:", a ** b)
```

OUTPUT

```
Addition: 19
Subtraction: 11
Multiplication: 60
Division: 3.75
Floor Division: 3
Modulus: 3
Exponent: 50625
```

5. Relational and Logical Operators

CODE

```
a, b = 10, 20
print(a > b, a < b, a == b, a != b)
print((a < b) and (b > 5))
print((a > b) or (b > 5))
print(not (a > b))
```

OUTPUT

```
False True False True
True
True
True
```

6. Type Conversion

CODE

```
x = "123"
y = int(x)
z = float(y)
s = str(z)
print(y, type(y))
print(z, type(z))
print(s, type(s))
```

OUTPUT

```
123 <class 'int'>
123.0 <class 'float'>
123.0 <class 'str'>
```

7. Swapping Two Numbers

CODE

```
a, b = 5, 10
print("Before:", a, b)
a, b = b, a
print("After:", a, b)
```

OUTPUT

```
Before: 5 10
After: 10 5
```

8. Simple Calculator

CODE

```
def calculator(a, b, op):
    if op == "+":
        return a + b
    elif op == "-":
        return a - b
    elif op == "*":
        return a * b
    elif op == "/":
        return a / b
    return None

print("10 + 5 =", calculator(10, 5, "+"))
print("10 - 5 =", calculator(10, 5, "-"))
print("10 * 5 =", calculator(10, 5, "*"))
print("10 / 5 =", calculator(10, 5, "/"))
```

OUTPUT

```
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2.0
```

9. Area and Perimeter of Rectangle

CODE

```
length, width = 8, 5
area = length * width
perimeter = 2 * (length + width)
print(f"Area = {area}")
print(f"Perimeter = {perimeter}")
```

OUTPUT

```
Area = 40
Perimeter = 26
```

10. Simple Interest Calculator

CODE

```
principal = 10000
rate = 7.5
time = 3
si = (principal * rate * time) / 100
print(f"Simple Interest = {si}")
```

OUTPUT

```
Simple Interest = 2250.0
```

Control Flow

11. Check Even or Odd

CODE

```
for num in [4, 7, 10, 15]:
    if num % 2 == 0:
        print(num, "is Even")
    else:
        print(num, "is Odd")
```

OUTPUT

```
4 is Even
7 is Odd
10 is Even
15 is Odd
```

12. Largest of Three Numbers

CODE

```
a, b, c = 12, 45, 30
largest = max(a, b, c)
print("Largest number is:", largest)
```

OUTPUT

```
Largest number is: 45
```

13. Leap Year Checker

CODE

```
def is_leap(year):  
    return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)  
  
for y in [2000, 2023, 2024, 1900]:  
    print(y, "is leap year:", is_leap(y))
```

OUTPUT

```
2000 is leap year: True  
2023 is leap year: False  
2024 is leap year: True  
1900 is leap year: False
```

14. Number Pattern using Nested Loops

CODE

```
rows = 5  
for i in range(1, rows + 1):  
    print(str(i) * i)
```

OUTPUT

```
1  
22  
333  
4444  
55555
```

15. Pyramid Pattern

CODE

```
rows = 5  
for i in range(1, rows + 1):  
    print(" " * (rows - i) + "*" * (2 * i - 1))
```

OUTPUT

```
  *  
 ***  
*****  
*****  
*****
```

16. While Loop - Sum of Digits

CODE

```
n = 12345
total = 0
while n > 0:
    total += n % 10
    n //= 10
print("Sum of digits =", total)
```

OUTPUT

```
Sum of digits = 15
```

17. Break and Continue Demo

CODE

```
for i in range(1, 11):
    if i == 5:
        continue
    if i == 9:
        break
    print(i, end=" ")
print()
```

OUTPUT

```
1 2 3 4 6 7 8
```

18. Multiplication Table

CODE

```
n = 7
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

OUTPUT

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Functions & Recursion

19. Function with Default Arguments

CODE

```
def greet(name, msg="Good Morning"):
    print(f"Hello {name}, {msg}")

greet("Prajyot")
greet("Team", "Welcome to CDAC")
```

OUTPUT

```
Hello Prajyot, Good Morning
Hello Team, Welcome to CDAC
```

20. Variable Arguments (*args, **kwargs)

CODE

```
def demo(*args, **kwargs):
    print("Positional:", args)
    print("Keyword:", kwargs)

demo(1, 2, 3, name="Prajyot", course="HPC")
```

OUTPUT

```
Positional: (1, 2, 3)
Keyword: {'name': 'Prajyot', 'course': 'HPC'}
```

21. Factorial using Recursion

CODE

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

for i in range(1, 7):
    print(f"{i}! = {factorial(i)}")
```

OUTPUT

```
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
```

22. Fibonacci using Recursion

CODE

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n - 1) + fib(n - 2)  
  
print([fib(i) for i in range(10)])
```

OUTPUT

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

23. Sum of Natural Numbers (Recursion)

CODE

```
def sum_n(n):  
    if n == 0:  
        return 0  
    return n + sum_n(n - 1)  
  
print("Sum of first 10 natural numbers:", sum_n(10))
```

OUTPUT

```
Sum of first 10 natural numbers: 55
```

24. GCD using Recursion (Euclidean Algorithm)

CODE

```
def gcd(a, b):  
    if b == 0:  
        return a  
    return gcd(b, a % b)  
  
print("GCD of 48 and 18 is:", gcd(48, 18))
```

OUTPUT

```
GCD of 48 and 18 is: 6
```

25. Power Function using Recursion

CODE

```
def power(base, exp):  
    if exp == 0:  
        return 1  
    return base * power(base, exp - 1)  
  
print("2^10 =", power(2, 10))
```

OUTPUT

```
2^10 = 1024
```

26. Tower of Hanoi

CODE

```
def hanoi(n, source, aux, target):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
    hanoi(n - 1, source, target, aux)
    print(f"Move disk {n} from {source} to {target}")
    hanoi(n - 1, aux, source, target)

hanoi(3, "A", "B", "C")
```

OUTPUT

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

27. Palindrome Check using Recursion

CODE

```
def is_palindrome(s):
    if len(s) <= 1:
        return True
    if s[0] != s[-1]:
        return False
    return is_palindrome(s[1:-1])

for word in ["madam", "python", "level"]:
    print(word, "->", is_palindrome(word))
```

OUTPUT

```
madam -> True
python -> False
level -> True
```

28. Higher Order Function Example

CODE

```
def apply_operation(func, x, y):
    return func(x, y)

print(apply_operation(lambda a, b: a + b, 5, 3))
print(apply_operation(lambda a, b: a * b, 5, 3))
```

OUTPUT

```
8
15
```

Strings

29. String Reversal

CODE

```
s = "Programming"  
print("Reversed:", s[::-1])
```

OUTPUT

```
Reversed: gnimmargorP
```

30. Palindrome String Check

CODE

```
def check_palindrome(s):  
    s = s.lower().replace(" ", "")  
    return s == s[::-1]  
  
print(check_palindrome("Madam"))  
print(check_palindrome("Hello"))
```

OUTPUT

```
True  
False
```

31. Count Vowels and Consonants

CODE

```
s = "Centre for Development of Advanced Computing"  
vowels = sum(1 for ch in s.lower() if ch in "aeiou")  
consonants = sum(1 for ch in s.lower() if ch.isalpha() and ch not in "aeiou")  
print("Vowels:", vowels)  
print("Consonants:", consonants)
```

OUTPUT

```
Vowels: 14  
Consonants: 25
```

32. String Formatting Methods

CODE

```
name, score = "Prajyot", 92.567  
print("Name: %s, Score: %.2f" % (name, score))  
print("Name: {}, Score: {:.2f}".format(name, score))  
print(f"Name: {name}, Score: {score:.2f}")
```

OUTPUT

```
Name: Prajyot, Score: 92.57  
Name: Prajyot, Score: 92.57  
Name: Prajyot, Score: 92.57
```

33. Word Frequency Counter

CODE

```
text = "data data is the new oil and oil is precious"
words = text.split()
freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1
for word, count in freq.items():
    print(f"{word}: {count}")
```

OUTPUT

```
data: 2
is: 2
the: 1
new: 1
oil: 2
and: 1
precious: 1
```

34. Anagram Checker

CODE

```
def is_anagram(a, b):
    return sorted(a.lower()) == sorted(b.lower())

print(is_anagram("listen", "silent"))
print(is_anagram("hello", "world"))
```

OUTPUT

```
True
False
```

35. Remove Duplicate Characters

CODE

```
s = "programming"
result = ""
for ch in s:
    if ch not in result:
        result += ch
print("After removing duplicates:", result)
```

OUTPUT

```
After removing duplicates: progamin
```

36. Capitalize Each Word (Title Case)

CODE

```
sentence = "cdac acts pune hpc training"  
print(sentence.title())
```

OUTPUT

```
Cdac Acts Pune Hpc Training
```

37. String Slicing Demo

CODE

```
s = "Python Programming"  
print("First 6 chars:", s[:6])  
print("Last 11 chars:", s[7:])  
print("Every 2nd char:", s[::2])  
print("Reversed:", s[::-1])
```

OUTPUT

```
First 6 chars: Python  
Last 11 chars: Programming  
Every 2nd char: Pto rgamn  
Reversed: gnimmargorP nohtyP
```

38. Longest Word in a Sentence

CODE

```
sentence = "High Performance Computing enables faster scientific simulations"  
words = sentence.split()  
longest = max(words, key=len)  
print("Longest word:", longest)
```

OUTPUT

```
Longest word: Performance
```

Data Structures (Built-in)

39. List Operations

CODE

```
nums = [5, 3, 8, 1, 9, 2]  
nums.append(20)  
nums.remove(3)  
nums.sort()  
print("Modified list:", nums)  
print("Max:", max(nums), "Min:", min(nums), "Sum:", sum(nums))
```

OUTPUT

```
Modified list: [1, 2, 5, 8, 9, 20]  
Max: 20 Min: 1 Sum: 45
```

40. List Comprehension - Squares

CODE

```
squares = [x**2 for x in range(1, 11)]
print(squares)
even_squares = [x**2 for x in range(1, 11) if x % 2 == 0]
print(even_squares)
```

OUTPUT

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
[4, 16, 36, 64, 100]
```

41. Nested List (Matrix) Operations

CODE

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
for row in matrix:
    print(row)
transpose = [[matrix[j][i] for j in range(3)] for i in range(3)]
print("Transpose:")
for row in transpose:
    print(row)
```

OUTPUT

```
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
Transpose:
[1, 4, 7]
[2, 5, 8]
[3, 6, 9]
```

42. Tuple Basics and Immutability

CODE

```
t = (10, 20, 30, "HPC")
print("Tuple:", t)
print("Length:", len(t))
a, b, c, d = t
print(a, b, c, d)
```

OUTPUT

```
Tuple: (10, 20, 30, 'HPC')
Length: 4
10 20 30 HPC
```

43. Dictionary Operations

CODE

```
student = {"name": "Prajyot", "course": "PGDHPC", "center": "CDAC Pune"}
student["year"] = 2024
print(student)
for key, value in student.items():
    print(f"{key}: {value}")
```

OUTPUT

```
{'name': 'Prajyot', 'course': 'PGDHPC', 'center': 'CDAC Pune', 'year': 2024}
name: Prajyot
course: PGDHPC
center: CDAC Pune
year: 2024
```

44. Set Operations

CODE

```
a = {1, 2, 3, 4, 5}
b = {4, 5, 6, 7, 8}
print("Union:", a | b)
print("Intersection:", a & b)
print("Difference:", a - b)
print("Symmetric Difference:", a ^ b)
```

OUTPUT

```
Union: {1, 2, 3, 4, 5, 6, 7, 8}
Intersection: {4, 5}
Difference: {1, 2, 3}
Symmetric Difference: {1, 2, 3, 6, 7, 8}
```

45. Merge Two Dictionaries

CODE

```
d1 = {"a": 1, "b": 2}
d2 = {"b": 3, "c": 4}
merged = {**d1, **d2}
print("Merged:", merged)
```

OUTPUT

```
Merged: {'a': 1, 'b': 3, 'c': 4}
```

46. List of Dictionaries - Sort by Key

CODE

```
students = [  
    {"name": "Amit", "marks": 85},  
    {"name": "Riya", "marks": 92},  
    {"name": "Sam", "marks": 78},  
]  
sorted_students = sorted(students, key=lambda s: s["marks"], reverse=True)  
for s in sorted_students:  
    print(s)
```

OUTPUT

```
{'name': 'Riya', 'marks': 92}  
{'name': 'Amit', 'marks': 85}  
{'name': 'Sam', 'marks': 78}
```

47. Find Duplicate Elements in a List

CODE

```
nums = [1, 2, 3, 4, 2, 5, 1, 6]  
seen = set()  
duplicates = set()  
for n in nums:  
    if n in seen:  
        duplicates.add(n)  
    seen.add(n)  
print("Duplicates:", duplicates)
```

OUTPUT

```
Duplicates: {1, 2}
```

48. Counter from collections

CODE

```
from collections import Counter  
text = "the quick brown fox jumps over the lazy dog the fox runs"  
count = Counter(text.split())  
print(count.most_common(3))
```

OUTPUT

```
[('the', 3), ('fox', 2), ('quick', 1)]
```

49. defaultdict Grouping Example

CODE

```
from collections import defaultdict
words = ["apple", "banana", "avocado", "blueberry", "cherry"]
groups = defaultdict(list)
for w in words:
    groups[w[0]].append(w)
for k, v in groups.items():
    print(k, "->", v)
```

OUTPUT

```
a -> ['apple', 'avocado']
b -> ['banana', 'blueberry']
c -> ['cherry']
```

50. Zip Function - Combine Two Lists

CODE

```
names = ["Prajyot", "Aman", "Riya"]
scores = [92, 85, 78]
for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

OUTPUT

```
Prajyot: 92
Aman: 85
Riya: 78
```

File Handling

51. Write and Read a Text File

CODE

```
with open("sample.txt", "w") as f:
    f.write("Line 1: Hello\n")
    f.write("Line 2: HPC Training at CDAC\n")

with open("sample.txt", "r") as f:
    print(f.read())
```

OUTPUT

```
Line 1: Hello
Line 2: HPC Training at CDAC
```

52. Append to a File

CODE

```
with open("sample.txt", "a") as f:  
    f.write("Line 3: Appended content\n")  
  
with open("sample.txt", "r") as f:  
    for line in f:  
        print(line.strip())
```

OUTPUT

```
Line 1: Hello  
Line 2: HPC Training at CDAC  
Line 3: Appended content
```

53. Count Lines, Words, Characters in a File

CODE

```
with open("sample.txt", "r") as f:  
    content = f.read()  
  
lines = content.splitlines()  
words = content.split()  
print("Lines:", len(lines))  
print("Words:", len(words))  
print("Characters:", len(content))
```

OUTPUT

```
Lines: 3  
Words: 13  
Characters: 68
```

54. Read File Line by Line with Line Numbers

CODE

```
with open("sample.txt", "r") as f:  
    for i, line in enumerate(f, start=1):  
        print(f"{i}: {line.strip()}")
```

OUTPUT

```
1: Line 1: Hello  
2: Line 2: HPC Training at CDAC  
3: Line 3: Appended content
```

55. Working with CSV Files

CODE

```
import csv

with open("data.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Marks"])
    writer.writerow(["Prajyot", 92])
    writer.writerow(["Aman", 85])

with open("data.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

OUTPUT

```
['Name', 'Marks']
['Prajyot', '92']
['Aman', '85']
```

56. Working with JSON Files

CODE

```
import json

data = {"name": "Prajyot", "skills": ["Python", "HPC", "Linux"], "year": 2024}
with open("data.json", "w") as f:
    json.dump(data, f, indent=2)

with open("data.json", "r") as f:
    loaded = json.load(f)
print(loaded)
```

OUTPUT

```
{'name': 'Prajyot', 'skills': ['Python', 'HPC', 'Linux'], 'year': 2024}
```

Exception Handling

57. Basic Try-Except

CODE

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print("Error caught:", e)
```

OUTPUT

```
Error caught: division by zero
```

58. Multiple Except Blocks

CODE

```
def safe_divide(a, b):  
    try:  
        return a / b  
    except ZeroDivisionError:  
        return "Cannot divide by zero"  
    except TypeError:  
        return "Invalid types provided"  
  
print(safe_divide(10, 2))  
print(safe_divide(10, 0))  
print(safe_divide(10, "a"))
```

OUTPUT

```
5.0  
Cannot divide by zero  
Invalid types provided
```

59. Try-Except-Else-Finally

CODE

```
def process(n):  
    try:  
        result = 100 / n  
    except ZeroDivisionError:  
        print("Cannot divide by zero!")  
    else:  
        print("Result:", result)  
    finally:  
        print("Execution completed for n =", n)  
  
process(5)  
process(0)
```

OUTPUT

```
Result: 20.0  
Execution completed for n = 5  
Cannot divide by zero!  
Execution completed for n = 0
```

60. Custom Exception Class

CODE

```
class InsufficientBalanceError(Exception):
    def __init__(self, balance, amount):
        super().__init__(f"Cannot withdraw {amount}, balance is only {balance}")

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientBalanceError(balance, amount)
    return balance - amount

try:
    withdraw(500, 1000)
except InsufficientBalanceError as e:
    print("Error:", e)
```

OUTPUT

```
Error: Cannot withdraw 1000, balance is only 500
```

61. Handling Multiple Exceptions in a Loop

CODE

```
values = ["10", "20", "abc", "0", "5"]
for v in values:
    try:
        num = int(v)
        print(100 / num)
    except ValueError:
        print(f"'{v}' is not a valid number")
    except ZeroDivisionError:
        print(f"Cannot divide by zero for value '{v}'")
```

OUTPUT

```
10.0
5.0
'abc' is not a valid number
Cannot divide by zero for value '0'
20.0
```

62. Assertions in Python

CODE

```
def calculate_average(marks):
    assert len(marks) > 0, "Marks list cannot be empty"
    return sum(marks) / len(marks)

print(calculate_average([80, 90, 70]))
try:
    calculate_average([])
except AssertionError as e:
    print("AssertionError:", e)
```

OUTPUT

```
80.0
AssertionError: Marks list cannot be empty
```

Object Oriented Programming

63. Class and Object Basics

CODE

```
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course

    def display(self):
        print(f"Name: {self.name}, Course: {self.course}")

s1 = Student("Prajyot", "PGDHPC")
s1.display()
```

OUTPUT

```
Name: Prajyot, Course: PGDHPC
```

64. Class Variables vs Instance Variables

CODE

```
class Counter:
    count = 0

    def __init__(self):
        Counter.count += 1

c1 = Counter()
c2 = Counter()
c3 = Counter()
print("Total instances created:", Counter.count)
```

OUTPUT

```
Total instances created: 3
```

65. Inheritance Example

CODE

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Dog(Animal):
    def speak(self):
        print("Dog barks")

class Cat(Animal):
    def speak(self):
        print("Cat meows")

for animal in [Animal(), Dog(), Cat()]:
    animal.speak()
```

OUTPUT

```
Animal makes a sound
Dog barks
Cat meows
```

66. Multiple Inheritance

CODE

```
class Father:
    def skills(self):
        print("Father: Gardening, Programming")

class Mother:
    def skills(self):
        print("Mother: Cooking, Painting")

class Child(Father, Mother):
    def skills(self):
        Father.skills(self)
        Mother.skills(self)
        print("Child: Sports")

Child().skills()
```

OUTPUT

```
Father: Gardening, Programming
Mother: Cooking, Painting
Child: Sports
```

67. Encapsulation with Private Variables

CODE

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

acc = BankAccount(1000)
acc.deposit(500)
print("Balance:", acc.get_balance())
```

OUTPUT

```
Balance: 1500
```

68. Polymorphism with Function Overriding

CODE

```
class Shape:
    def area(self):
        return 0

class Rectangle(Shape):
    def __init__(self, l, w):
        self.l, self.w = l, w

    def area(self):
        return self.l * self.w

class Circle(Shape):
    def __init__(self, r):
        self.r = r

    def area(self):
        return 3.14159 * self.r ** 2

shapes = [Rectangle(4, 5), Circle(3)]
for s in shapes:
    print(f"{type(s).__name__} area: {s.area():.2f}")
```

OUTPUT

```
Rectangle area: 20.00
Circle area: 28.27
```

69. Abstract Base Class

CODE

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):
    def start(self):
        print("Car started with a key")

class Bike(Vehicle):
    def start(self):
        print("Bike started with a kick")

for v in [Car(), Bike()]:
    v.start()
```

OUTPUT

```
Car started with a key
Bike started with a kick
```

70. Operator Overloading

CODE

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"

v1 = Vector(2, 3)
v2 = Vector(4, 5)
print("v1 + v2 =", v1 + v2)
```

OUTPUT

```
v1 + v2 = (6, 8)
```

71. Static Method and Class Method

CODE

```
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b

    @classmethod
    def description(cls):
        return f"This is {cls.__name__} utility class"

print(MathUtils.add(5, 7))
print(MathUtils.description())
```

OUTPUT

```
12
This is MathUtils utility class
```

72. Property Decorator (Getter/Setter)

CODE

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def fahrenheit(self):
        return (self._celsius * 9/5) + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        self._celsius = (value - 32) * 5/9

t = Temperature(25)
print("Fahrenheit:", t.fahrenheit)
t.fahrenheit = 98.6
print("Celsius after setting F:", round(t._celsius, 2))
```

OUTPUT

```
Fahrenheit: 77.0
Celsius after setting F: 37.0
```

73. Duck Typing Demonstration

CODE

```
class Duck:
    def sound(self):
        print("Quack Quack")

class Human:
    def sound(self):
        print("I can imitate a duck: Quack!")

def make_it_quack(entity):
    entity.sound()

make_it_quack(Duck())
make_it_quack(Human())
```

OUTPUT

```
Quack Quack
I can imitate a duck: Quack!
```

74. Composition Example (Has-A Relationship)

CODE

```
class Engine:
    def start(self):
        print("Engine starting...")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start_car(self):
        self.engine.start()
        print("Car is ready to drive")

Car().start_car()
```

OUTPUT

```
Engine starting...
Car is ready to drive
```

Iterators, Generators & Decorators

75. Custom Iterator Class

CODE

```
class CountUpTo:
    def __init__(self, limit):
        self.limit = limit
        self.current = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.current < self.limit:
            self.current += 1
            return self.current
        raise StopIteration

for num in CountUpTo(5):
    print(num, end=" ")
print()
```

OUTPUT

```
1 2 3 4 5
```

76. Generator Function - Fibonacci Sequence

CODE

```
def fibonacci_gen(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b

print(list(fibonacci_gen(10)))
```

OUTPUT

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

77. Generator Expression

CODE

```
squares = (x**2 for x in range(1, 6))
print(list(squares))
total = sum(x**2 for x in range(1, 6))
print("Sum of squares:", total)
```

OUTPUT

```
[1, 4, 9, 16, 25]
Sum of squares: 55
```

78. Simple Function Decorator

CODE

```
def logger(func):
    def wrapper(*args, **kwargs):
        print(f"Calling function: {func.__name__}")
        result = func(*args, **kwargs)
        print(f"Finished: {func.__name__}")
        return result
    return wrapper

@logger
def add(a, b):
    return a + b

print("Result:", add(5, 3))
```

OUTPUT

```
Calling function: add
Finished: add
Result: 8
```

79. Decorator with Arguments

CODE

```
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                func(*args, **kwargs)
            return wrapper
        return decorator

@repeat(3)
def greet():
    print("Hello from CDAC!")

greet()
```

OUTPUT

```
Hello from CDAC!
Hello from CDAC!
Hello from CDAC!
```

80. Timing Decorator

CODE

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end - start:.6f} seconds")
        return result
    return wrapper

@timer
def compute_sum(n):
    return sum(range(n))

print("Sum:", compute_sum(1000000))
```

OUTPUT

```
compute_sum took 0.013415 seconds
Sum: 499999500000
```

81. Context Manager using Class

CODE

```
class FileManager:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode

    def __enter__(self):
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()
        print("File closed automatically")

with FileManager("context_demo.txt", "w") as f:
    f.write("Written using custom context manager")
print("Done")
```

OUTPUT

```
File closed automatically
Done
```

82. Context Manager using contextlib

CODE

```
from contextlib import contextmanager

@contextmanager
def open_resource(name):
    print(f"Opening {name}")
    yield name
    print(f"Closing {name}")

with open_resource("Database Connection") as res:
    print(f"Using {res}")
```

OUTPUT

```
Opening Database Connection
Using Database Connection
Closing Database Connection
```

Functional Programming

83. Lambda Functions

CODE

```
square = lambda x: x ** 2
add = lambda a, b: a + b
print(square(5))
print(add(3, 4))
```

OUTPUT

```
25
7
```

84. Map Function

CODE

```
nums = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x ** 2, nums))
print(squared)
```

OUTPUT

```
[1, 4, 9, 16, 25]
```

85. Filter Function

CODE

```
nums = list(range(1, 21))
evens = list(filter(lambda x: x % 2 == 0, nums))
print("Even numbers:", evens)
```

OUTPUT

```
Even numbers: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

86. Reduce Function

CODE

```
from functools import reduce
nums = [1, 2, 3, 4, 5]
product = reduce(lambda a, b: a * b, nums)
print("Product of list:", product)
```

OUTPUT

```
Product of list: 120
```

87. functools.lru_cache for Memoization

CODE

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

print([fib(i) for i in range(15)])
```

OUTPUT

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
```

88. itertools.permutations and combinations

CODE

```
from itertools import permutations, combinations

data = ["A", "B", "C"]
print("Permutations:", list(permutations(data, 2)))
print("Combinations:", list(combinations(data, 2)))
```

OUTPUT

```
Permutations: [('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'C'), ('C', 'A'), ('C', 'B')]
Combinations: [('A', 'B'), ('A', 'C'), ('B', 'C')]
```

89. itertools.groupby

CODE

```
from itertools import groupby

data = [("fruit", "apple"), ("fruit", "banana"), ("veg", "carrot"), ("veg", "potato")]
for key, group in groupby(data, key=lambda x: x[0]):
    print(key, "->", [item[1] for item in group])
```

OUTPUT

```
fruit -> ['apple', 'banana']
veg -> ['carrot', 'potato']
```

90. Partial Functions with functools

CODE

```
from functools import partial

def power(base, exponent):
    return base ** exponent

square = partial(power, exponent=2)
cube = partial(power, exponent=3)
print("Square of 5:", square(5))
print("Cube of 3:", cube(3))
```

OUTPUT

```
Square of 5: 25
Cube of 3: 27
```

Standard Library Modules

91. Math Module Basics

CODE

```
import math
print("sqrt(16):", math.sqrt(16))
print("factorial(5):", math.factorial(5))
print("pi:", math.pi)
print("ceil(4.2):", math.ceil(4.2), "floor(4.8):", math.floor(4.8))
```

OUTPUT

```
sqrt(16): 4.0
factorial(5): 120
pi: 3.141592653589793
ceil(4.2): 5 floor(4.8): 4
```

92. Random Module (with fixed seed)

CODE

```
import random
random.seed(42)
print("Random int (1-100):", random.randint(1, 100))
print("Random choice:", random.choice(["A", "B", "C", "D"]))
sample_list = [1, 2, 3, 4, 5]
random.shuffle(sample_list)
print("Shuffled list:", sample_list)
```

OUTPUT

```
Random int (1-100): 82
Random choice: A
Shuffled list: [2, 4, 5, 3, 1]
```

93. Datetime Module

CODE

```
from datetime import datetime, timedelta

now = datetime(2024, 1, 15, 10, 30)
print("Current-like date:", now)
future = now + timedelta(days=30)
print("30 days later:", future)
print("Formatted:", now.strftime("%d-%m-%Y %H:%M"))
```

OUTPUT

```
Current-like date: 2024-01-15 10:30:00
30 days later: 2024-02-14 10:30:00
Formatted: 15-01-2024 10:30
```

94. Regular Expressions - Pattern Matching

CODE

```
import re
text = "Contact us at support@cdac.in or admin@cdac.in for HPC queries"
emails = re.findall(r"[\w.]+@[ \w.]+", text)
print("Emails found:", emails)
```

OUTPUT

```
Emails found: ['support@cdac.in', 'admin@cdac.in']
```

95. Regular Expressions - Validate Phone Number

CODE

```
import re

def is_valid_phone(number):
    pattern = r"^[6-9]\d{9}$"
    return bool(re.match(pattern, number))

for num in ["9876543210", "1234567890", "98765432"]:
    print(num, "->", is_valid_phone(num))
```

OUTPUT

```
9876543210 -> True
1234567890 -> False
98765432 -> False
```

96. OS Module - File System Info

CODE

```
import os
print("Current Working Directory:", os.getcwd())
print("Files in directory:", [f for f in os.listdir(".") if f.endswith(".txt")][:5])
```

OUTPUT

```
Current Working Directory: /home/claude/pyprogs
Files in directory: ['sample.txt', 'context_demo.txt']
```

97. Working with Collections - namedtuple

CODE

```
from collections import namedtuple

Point = namedtuple("Point", ["x", "y"])
p1 = Point(3, 4)
print(p1)
print("Distance from origin:", (p1.x ** 2 + p1.y ** 2) ** 0.5)
```

OUTPUT

```
Point(x=3, y=4)
Distance from origin: 5.0
```

Sorting Algorithms

98. Bubble Sort

CODE

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

print(bubble_sort([64, 34, 25, 12, 22, 11, 90]))
```

OUTPUT

```
[11, 12, 22, 25, 34, 64, 90]
```

99. Selection Sort

CODE

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

print(selection_sort([29, 10, 14, 37, 14]))
```

OUTPUT

```
[10, 14, 14, 29, 37]
```

100. Insertion Sort

CODE

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

print(insertion_sort([12, 11, 13, 5, 6]))
```

OUTPUT

```
[5, 6, 11, 12, 13]
```

101. Merge Sort

CODE

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

print(merge_sort([38, 27, 43, 3, 9, 82, 10]))
```

OUTPUT

```
[3, 9, 10, 27, 38, 43, 82]
```

102. Quick Sort

CODE

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)

print(quick_sort([10, 7, 8, 9, 1, 5]))
```

OUTPUT

```
[1, 5, 7, 8, 9, 10]
```

103. Counting Sort

CODE

```
def counting_sort(arr):
    max_val = max(arr)
    count = [0] * (max_val + 1)
    for num in arr:
        count[num] += 1
    result = []
    for i, c in enumerate(count):
        result.extend([i] * c)
    return result

print(counting_sort([4, 2, 2, 8, 3, 3, 1]))
```

OUTPUT

```
[1, 2, 2, 3, 3, 4, 8]
```

Searching Algorithms

104. Linear Search

CODE

```
def linear_search(arr, target):
    for i, val in enumerate(arr):
        if val == target:
            return i
    return -1

arr = [5, 8, 2, 9, 1, 7]
print("Index of 9:", linear_search(arr, 9))
print("Index of 100:", linear_search(arr, 100))
```

OUTPUT

```
Index of 9: 3
Index of 100: -1
```

105. Binary Search (Iterative)

CODE

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

arr = sorted([5, 8, 2, 9, 1, 7, 12, 15])
print("Sorted array:", arr)
print("Index of 12:", binary_search(arr, 12))
```

OUTPUT

```
Sorted array: [1, 2, 5, 7, 8, 9, 12, 15]
Index of 12: 6
```

106. Binary Search (Recursive)

CODE

```
def binary_search_rec(arr, target, low, high):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_rec(arr, target, mid + 1, high)
    else:
        return binary_search_rec(arr, target, low, mid - 1)

arr = [1, 3, 5, 7, 9, 11, 13]
print("Index of 7:", binary_search_rec(arr, 7, 0, len(arr) - 1))
```

OUTPUT

```
Index of 7: 3
```

Data Structures & Algorithms

107. Stack Implementation

CODE

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop() if self.items else None

    def peek(self):
        return self.items[-1] if self.items else None

s = Stack()
s.push(10); s.push(20); s.push(30)
print("Top element:", s.peek())
print("Popped:", s.pop())
print("Stack after pop:", s.items)
```

OUTPUT

```
Top element: 30
Popped: 30
Stack after pop: [10, 20]
```

108. Queue Implementation

CODE

```
from collections import deque

queue = deque()
queue.append(1)
queue.append(2)
queue.append(3)
print("Queue:", list(queue))
print("Dequeued:", queue.popleft())
print("Queue after dequeue:", list(queue))
```

OUTPUT

```
Queue: [1, 2, 3]
Dequeued: 1
Queue after dequeue: [2, 3]
```

109. Singly Linked List Implementation

CODE

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        curr = self.head
        while curr.next:
            curr = curr.next
        curr.next = new_node

    def display(self):
        curr = self.head
        result = []
        while curr:
            result.append(curr.data)
            curr = curr.next
        print(" -> ".join(map(str, result)))

ll = LinkedList()
for val in [10, 20, 30, 40]:
    ll.append(val)
ll.display()
```

OUTPUT

```
10 -> 20 -> 30 -> 40
```

110. Binary Search Tree - Insert and Inorder Traversal

CODE

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root

def inorder(root, result):
    if root:
        inorder(root.left, result)
        result.append(root.key)
        inorder(root.right, result)
    return result

root = None
for val in [50, 30, 70, 20, 40, 60, 80]:
    root = insert(root, val)

print("Inorder traversal (sorted):", inorder(root, []))
```

OUTPUT

```
Inorder traversal (sorted): [20, 30, 40, 50, 60, 70, 80]
```

111. Graph - BFS Traversal

CODE

```
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"],
    "E": ["B", "F"],
    "F": ["C", "E"],
}

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order

print("BFS from A:", bfs(graph, "A"))
```

OUTPUT

```
BFS from A: ['A', 'B', 'C', 'D', 'E', 'F']
```

112. Graph - DFS Traversal

CODE

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"],
    "E": ["B", "F"],
    "F": ["C", "E"],
}

def dfs(graph, node, visited=None, order=None):
    if visited is None:
        visited = set()
        order = []
    visited.add(node)
    order.append(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited, order)
    return order

print("DFS from A:", dfs(graph, "A"))
```

OUTPUT

```
DFS from A: ['A', 'B', 'D', 'E', 'F', 'C']
```

Dynamic Programming

113. Fibonacci using Dynamic Programming (Tabulation)

CODE

```
def fib_dp(n):
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp

print(fib_dp(15))
```

OUTPUT

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610]
```

114. 0/1 Knapsack Problem

CODE

```
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]], dp[i - 1][w])
            else:
                dp[i][w] = dp[i - 1][w]
    return dp[n][capacity]

weights = [1, 3, 4, 5]
values = [1, 4, 5, 7]
print("Max value in knapsack:", knapsack(weights, values, 7))
```

OUTPUT

```
Max value in knapsack: 9
```

115. Longest Common Subsequence

CODE

```
def lcs(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]

print("LCS length of 'ABCBADAB' and 'BDCABA':", lcs("ABCBADAB", "BDCABA"))
```

OUTPUT

```
LCS length of 'ABCBADAB' and 'BDCABA': 4
```

116. Coin Change Problem (Minimum Coins)

CODE

```
def coin_change(coins, amount):
    dp = [float("inf")] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for x in range(coin, amount + 1):
            dp[x] = min(dp[x], dp[x - coin] + 1)
    return dp[amount] if dp[amount] != float("inf") else -1

print("Minimum coins needed for 11 using [1,2,5]:", coin_change([1, 2, 5], 11))
```

OUTPUT

```
Minimum coins needed for 11 using [1,2,5]: 3
```

Advanced Python Concepts

117. Closures in Python

CODE

```
def make_multiplier(factor):
    def multiplier(number):
        return number * factor
    return multiplier

double = make_multiplier(2)
triple = make_multiplier(3)
print("Double of 5:", double(5))
print("Triple of 5:", triple(5))
```

OUTPUT

```
Double of 5: 10
Triple of 5: 15
```

118. *args, **kwargs in Class Constructor

CODE

```
class Product:
    def __init__(self, name, **specs):
        self.name = name
        self.specs = specs

    def display(self):
        print(f"Product: {self.name}")
        for key, val in self.specs.items():
            print(f"  {key}: {val}")

p = Product("Laptop", ram="16GB", cpu="i7", storage="512GB SSD")
p.display()
```

OUTPUT

```
Product: Laptop
  ram: 16GB
  cpu: i7
  storage: 512GB SSD
```

119. Multithreading Example

CODE

```
import threading

def print_numbers():
    for i in range(1, 6):
        print(f"Number: {i}")

def print_letters():
    for letter in "ABCDE":
        print(f"Letter: {letter}")

t1 = threading.Thread(target=print_numbers)
t2 = threading.Thread(target=print_letters)
t1.start(); t1.join()
t2.start(); t2.join()
print("Both threads finished")
```

OUTPUT

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
Letter: A
Letter: B
Letter: C
Letter: D
Letter: E
Both threads finished
```

120. Multiprocessing Example

CODE

```
from multiprocessing import Pool

def square(n):
    return n * n

if __name__ == "__main__":
    with Pool(4) as p:
        results = p.map(square, [1, 2, 3, 4, 5])
    print("Squares computed using multiprocessing:", results)
```

OUTPUT

```
Squares computed using multiprocessing: [1, 4, 9, 16, 25]
```

121. Async/Await Example

CODE

```
import asyncio

async def fetch_data(name, delay):
    print(f"Start fetching {name}")
    await asyncio.sleep(delay)
    print(f"Finished fetching {name}")
    return f"{name}-data"

async def main():
    results = await asyncio.gather(
        fetch_data("A", 0.2),
        fetch_data("B", 0.1),
    )
    print("Results:", results)

asyncio.run(main())
```

OUTPUT

```
Start fetching A
Start fetching B
Finished fetching B
Finished fetching A
Results: ['A-data', 'B-data']
```

122. Metaclass Example

CODE

```
class Meta(type):
    def __new__(mcs, name, bases, namespace):
        namespace["created_by"] = "Meta"
        return super().__new__(mcs, name, bases, namespace)

class MyClass(metaclass=Meta):
    pass

print(MyClass.created_by)
```

OUTPUT

Meta

123. Data Classes (Python 3.7+)

CODE

```
from dataclasses import dataclass

@dataclass
class Employee:
    name: str
    role: str
    salary: float

e = Employee("Prajyot", "System Administrator", 55000.0)
print(e)
print("Salary:", e.salary)
```

OUTPUT

Employee(name='Prajyot', role='System Administrator', salary=55000.0)
Salary: 55000.0

124. Enum Class Example

CODE

```
from enum import Enum

class Status(Enum):
    PENDING = 1
    RUNNING = 2
    COMPLETED = 3

job_status = Status.RUNNING
print(job_status)
print(job_status.name, job_status.value)
```

OUTPUT

Status.RUNNING
RUNNING 2

125. Type Hints and mypy-style Annotations

CODE

```
def add_numbers(a: int, b: int) -> int:
    return a + b

def greet(name: str) -> str:
    return f"Hello, {name}!"

print(add_numbers(5, 7))
print(greet("Prajyot"))
```

OUTPUT

```
12
Hello, Prajyot!
```